Building together

The DiracX Transformation System

DiracX hackathon · 12th DIRAC(X) Users' Workshop · FZU Prague



From a game to working code

TUE 16:10

Ice-breaker game

40 min

Run a workgraph by hand, with sticky notes.

TUE 16:50

1-2-4-All

1 h

What must exist, and how do we split the work?

WED 14:00

Mob programming

1 h 50

We build the database and the state machines together, with Claude.

WED 16:10

Planning poker

45 min

We size the work items and form pairs.

WED 17:00 → FRI

We build

about 6 h

Pairs and small pull requests. Demo on Friday.

> Code is becoming cheap.
> What we need is a **common understanding** of what to build, and of how we **review** it.

1 · Ice-breaker game

Run a Monte Carlo workgraph by hand

The game: 8 seeds become 1 merged file

Seed stack
8 red notes, S1 … S8

→

SIMULATION
1 seed per parcel

→

RECONSTRUCTION
2 green per parcel

→

MERGE
4 blue per parcel

→

Finished
1 taped strip: shout **“Completed!”**

ON EVERY BOARD, EACH PARCEL MAKES THE SAME TRIP

one per transformation: the board's player

shared by all transformations

Feeder → Pool → **Packer** → Ready → **Dispatcher** → **Compute backend** → **Completion** → ✓ Done
new notes → Outputs

✗ the note goes back to the Pool, a **new** parcel takes it

dark a player dashed a place on the table 6 players: one per board, plus Dispatcher, Compute backend, Completion

Round 1

ROUND

1

7 min

- Read your sheet. Ask your team, not us.
- Start when we say **go**.
- 8 seeds → 8 green → 4 blue → **1 taped strip**.
- Shout **“Completed!”** when it is on the table.

Round 2

ROUND

2

7 min

- Reset the table: notes back in the stack, new parcels.
- **Same game, same goal.**
- Shout **“Completed!”** when the strip is on the table.

Round 3

ROUND

3

7 min

- **Change seats:** move one seat to the left and take the sheet in front of you.
- Reconstruction receives a **new version of its packer plugin**.
- Same goal. Shout **“Completed!”**

Debrief

What happened

- Where did you **wait**? Who was the bottleneck?
- When a job failed, what was **retried**: the parcel or the seed?
- How did you know the work was **finished**?
- Who knew how many notes were where? Now imagine **100 million** notes.

YOU PLAYED	WE WILL BUILD
Board players: feeding	**Feeder, edge feeder**
Board players: packing	**Packer**
Dispatcher, Compute backend	**Dispatcher, dummy backend**
Completion	**Completion: done, failed, split**
Nobody kept the books	**Database, state machines, counters**
The rules on the boards	**CWL submission**

> In the game, you moved the notes yourselves. In DiracX, a feeder or a packer only **says** what should happen. The core then updates the database in one step, so a note is never lost or counted twice.

2 · 1-2-4-All

What must exist, and how do we split it?

1-2-4-All

> What must exist so that `simulation.cwl` runs on a dummy backend at the end of the sprint? What is unclear?

1

`5 min · alone`

One piece of work per note 🟨, one question per note 🟥. In silence.

2

`10 min · in pairs`

Merge your notes, remove duplicates, sharpen the words.

4

`15 min · in fours`

Group the notes into **work items** a pair can finish in 2 to 3 days. Put them in order.

All

`25 min · together`

One list of work items for this week and the next 2 to 3 sprints, in order. Each group adds only what is new.

Outcome: our backlog. The first sprint in detail, the next ones roughly. A good item ends with a demo test and names what it depends on. Planning poker sizes the first items on Wednesday.

3 · Mob programming

The shared core, built together

Mob programming: the shared core, with Claude

WHAT WE BUILD

- database tables · DX-ADR-004
- state machines as data · DX-ADR-005
- transitions module: the only code that changes a status
- one transition + its test

1 · THE ROOM
Decides the next prompt
architects check it against the ADRs

→

2 · ONE PERSON
Types it into Claude
on the projector

→

3 · GIT
One prompt, one commit
pushed to a draft pull request

→

4 · EVERYONE
Reviews the diff live
and comments on GitHub

the comments become the next prompt

> Every pair will build on this code, so we learn it together.

4 · Planning poker

How big is each item?

Planning poker

1	2	3	5	8	13	?
Trivial. We know exactly what to change.	Small and clear.	Medium. Some thinking, few unknowns.	Large. Several parts, some unknowns.	Very large. Try to split it.	Too big. Split it before we start.	I do not understand the item.
 1 finger	 2 fingers	 3 fingers	 one hand	 5 + 3	 arms crossed	 fist

- Points measure **size**: effort, complexity, uncertainty. Not hours.
- “3, 2, 1, **show**”: everyone shows their fingers at the same time.
- The **highest** and the **lowest** explain. Then one more vote.
- Our usual pace: about **6 points per person per sprint**.

Now, let’s build it

- Pick your first item
- Small pull requests to `main`
- Review each other's pull requests
- Demo on Friday

Backup

Game material: set-up, table, role sheets

Game set-up and timing

Per team of 6 to 8

- Small sticky notes: one red, one green, one blue pad
- 8 red notes written in advance: `S1` … `S8`
- Tape, pens, scrap paper cut in quarters (the parcels)
- 3 boards, 6 role sheets, the Backend's card for round 2
- 7 players: second Compute backend. 8: Reconstruction split into feeder and packer.

TIME	STEP
3 min	**Explain the game**
7 min	**Read sheets, set up the table**
7 min	**Round 1, happy path**
2 + 7 min	**Reset, round 2, a surprise failure**
2 + 7 min	**Reset, round 3, new seats and new packer rule**
8 min	**Debrief**

The table

SIMULATION board	RECONSTRUCTION board	MERGE board
1 red per parcel	2 green per parcel	4 blue per parcel
Pool / Done / Outputs	Pool / Done / Outputs	Pool / Done / Outputs

Seed stack
S1 … S8, written in advance

Ready
parcels waiting for the Dispatcher

Backend seat
green and blue pads, tape

Bin
every used parcel, done or failed. Never reused.

A parcel is a quarter sheet with the input notes stuck on it (`P5 · RECO`). After the job: ✓ inputs go to Done, ✗ inputs go back to their Pool, the empty parcel goes to the Bin.

A note's name says where it comes from: `G3` is the green file made from seed `S3` ; `B1 = G3 + G5` .

Simulation

You play the Simulation transformation's two plugins: its **feeder** and its **packer**. Each transformation has its own.

You hold: the seed stack S1 … S8, paper quarters, a pen.

1. **Feed:** put seeds from the stack into your Pool, two or three at a time.
2. When the stack is empty, say aloud: **“Simulation feed is over.”**
3. **Pack:** 1 seed per parcel. Write `P1 · SIM` (then P2…) on a paper quarter, stick the seed on it, put it on **Ready**.

Reconstruction

You play the Reconstruction transformation's feeder and packer. Your feeder is an **edge feeder**: its inputs are the outputs of Simulation.

You hold: paper quarters, a pen. Your packer rule may change in round 3.

1. **Feed:** move each 🟩 green note from the Simulation Outputs into your Pool.
2. When Simulation's feed is over and nothing of Simulation is left (pool, Ready, backend, outputs), say: **“Reconstruction feed is over.”**
3. **Pack:** 2 green per parcel (`P7 · RECO`), on Ready. Not enough notes? Wait. Once your feed is over, pack what is left.

Merge

You play the Merge transformation's feeder and packer. Your feeder is an **edge feeder**: its inputs are the outputs of Reconstruction.

You hold: paper quarters, a pen.

1. **Feed:** move each 🟦 blue note from the Reconstruction Outputs into your Pool.
2. When Reconstruction's feed is over and nothing of Reconstruction is left, say: **“Merge feed is over.”**
3. **Pack:** 4 blue per parcel (`P12 · MERGE`), on Ready.

   Not enough notes? Wait. Once your feed is over, pack what is left.

Dispatcher

In the real system: the part that hands each parcel to a backend. There can be several backends; the dispatcher chooses one.

You hold: nothing.

1. Take the oldest parcel on **Ready**.
2. Give it to a Compute backend that is **free**: no parcel in its hands.
3. If every backend is busy, wait with the parcel. Never give a backend two parcels.

Compute backend

In the real system: where jobs run. All three transformations send their jobs to you; the boards only keep the books.

You hold: the green and blue pads, tape, a pen. If you get a card, read it in secret.

1. **SIM** parcel: for seed `S3`, write a green `G3` and stick it on the parcel.
2. **RECO** parcel: for two green notes, write one blue `B1 = G3 + G5`.
3. **MERGE** parcel: tape the four blue notes into one strip.
4. Write ✓ next to each input that worked (✗ if not), then give the parcel to the Completion.

Completion

In the real system: the step that reads the job's report and decides what happens to each input.

You hold: nothing.

1. ✓ input: put it in the **Done** column of its board. Put the new notes in **Outputs**.
2. ✗ input: put it back in the **pool** of its board. The Packer will make a new parcel.
3. Put the empty parcel in the **Bin**. A parcel is never reused.
4. When the taped strip arrives and every pool and Ready are empty, shout **“Completed!”**